Honeypot Behavior Tracker - Progress Report

Hayden Stilley
Network Security Semester Project - Progress Report
Final Deliverables:

- SSH Listener: implemented → **honeypot.py**
- Session Monitor: basic per-client thread management in place
- Logger: prints to console currently, will have logging function soon
- Report Generator: not yet implemented
- Attacker Emulation: early-stage client script → **adversary.py**.

SSH Listener is implemented in **honeypot.py**. **Session Monitor, Logger,** and **Report Generator** will be implemented in **honeypot.py** as well. **Attacker Emulation** is in an early stage of implementation, handled by client script **adversary.py**.

honeypot.py

Standard simulation of an SSH server, implemented in Python with **paramiko** library. Implementation is based on the standard server pattern provided by **paramiko**'s `ServerInterface`, which is largely comparable to industry standard boilerplate for SSH connection handling.
This program creates sockets, binds them to hosts and ports, and handles transport negotiation and fake authentication prompts to goad attackers into connecting to what they believe to be a legitimate SSH server.
When script initializes, **honeypot.py** runs **start_honeypot(host, port)**, logs an incoming connection and its corresponding client credentials attempted by **adversary.py**.

```python
class SSHHoneypot(paramiko.ServerInterface):

    def check_auth_password(self, username, password):
        # Log credentials attempted
        print(f"[{datetime.now()}] Attempted login: {usernam
        return paramiko.AUTH_FAILED  # never allow login

def handle_client(client_socket, addr):
    try:
        transport = paramiko.Transport(client_socket)
        transport.add_server_key(HOST_KEY)
        server = SSHHoneypot()
        try:
            transport.start_server(server=server)
        except paramiko.SSHException:
            print(f"[{datetime.now()}] SSH negotiation faile
            return

        # Wait for a channel
        chan = transport.accept(20)
        if chan is not None:
            chan.close()
        transport.close()
    except Exception as e:
        print(f"[{datetime.now()}] Exception: {e}")
    finally:
        client_socket.close()
```

See **Figure 1** for a demonstration of **honeypot.py** and **adversary.py** in action.

start_honeypot(host, port):

The program's de-facto main method. Starts by creating a socket, then binds a host and a port to the socket. It then starts its listener and enters primary loop, blocking until a client connects or the session times out.
When a client connects, program creates and initiates a thread for the client.

```
Command Line Argument Defaults:
    host='0.0.0.0'
    port=2222
Assign server socket
Bind socket to (host, port)
Start listening for clients
Begin primary program loop:
    Block until client connects
```

```
    Accept client and address at socket
    Start thread for client by running **handle-client(client_socket, addr)**
```

adversary.py

An emulation of an attack or a set of attacks against the honeypot server. Will add behavior to emulate attack archetypes for analysis by report generator.

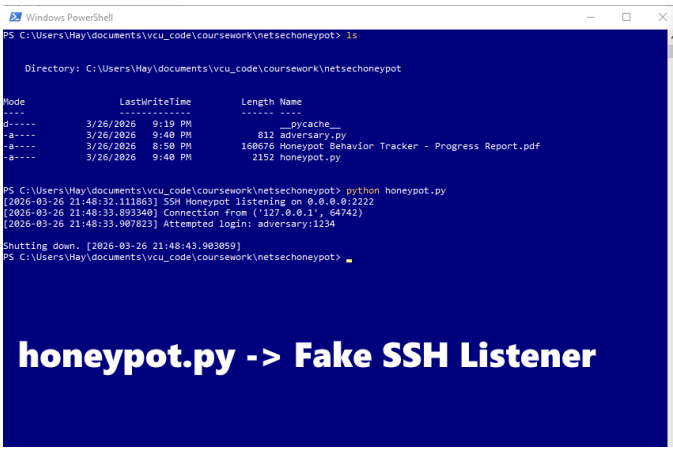
adversary.py 1, U

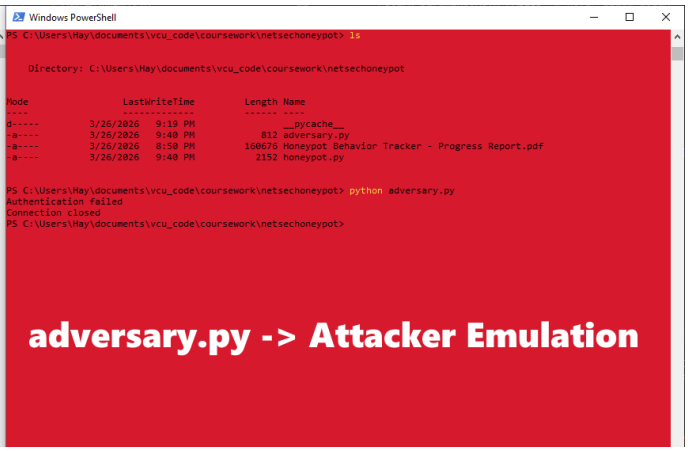
NetsecHoneypot > adversary.py > ssh_connect

```python
import paramiko

def ssh_connect(host='127.0.0.1', port=2222, username='adver
    try:
        # Create SSH client
        client = paramiko.SSHClient()
        client.set_missing_host_key_policy(paramiko.AutoAddP

        # Connect to the server
        client.connect(hostname=host, port=port, username=us
        print(f"Connected to {host}:{port} as {username}")


        print("Command output:", stdout.read().decode().stri


    except paramiko.AuthenticationException:
        print("Authentication failed")
    except paramiko.SSHException as e:
        print(f"SSH Error: {e}")
    finally:
        client.close()
        print("Connection closed")

if __name__ == "__main__":
    ssh_connect()
```

See **Figure 1** for a demonstration of **honeypot.py** and **adversary.py** in action.

Figure 1


Windows PowerShell
PS C:\Users\Hay\documents\vcu_code\coursework\netsechoneypot> ls
Directory: C:\Users\Hay\documents\vcu_code\coursework\netsechoneypot
Mode LastWriteTime Length Name
---- ---------- ------ ----
d---- 3/26/2026 9:19 PM __pycache__
-a---- 3/26/2026 9:40 PM 812 adversary.py
-a---- 3/26/2026 8:50 PM 160676 Honeypot Behavior Tracker - Progress Report.pdf
-a---- 3/26/2026 9:40 PM 2152 honeypot.py
PS C:\Users\Hay\documents\vcu_code\coursework\netsechoneypot> python honeypot.py
[2026-03-26 21:48:32.111863] SSH Honeypot listening on 0.0.0.0:2222
[2026-03-26 21:48:33.893340] Connection from ('127.0.0.1', 64742)
[2026-03-26 21:48:33.907823] Attempted login: adversary:1234
Shutting down. [2026-03-26 21:48:43.903059]
PS C:\Users\Hay\documents\vcu_code\coursework\netsechoneypot>
honeypot.py -> Fake SSH Listener


Windows PowerShell
PS C:\Users\Hay\documents\vcu_code\coursework\netsechoneypot> ls
Directory: C:\Users\Hay\documents\vcu_code\coursework\netsechoneypot
Mode LastWriteTime Length Name
---- ---------- ------ ----
d---- 3/26/2026 9:19 PM __pycache__
-a---- 3/26/2026 9:40 PM 812 adversary.py
-a---- 3/26/2026 8:50 PM 160676 Honeypot Behavior Tracker - Progress Report.pdf
-a---- 3/26/2026 9:40 PM 2152 honeypot.py
PS C:\Users\Hay\documents\vcu_code\coursework\netsechoneypot> python adversary.py
Authentication failed
Connection closed
PS C:\Users\Hay\documents\vcu_code\coursework\netsechoneypot>
adversary.py -> Attacker Emulation